



INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus



**Advanced Visual Programming
Lab**

Assignment No 02

Submitted To:

Sir Uzair

Submitted By:

Momina Waqar

232201039

BSCS 6 A

BS in Computer Science,

K.I.C.S.I.T , Kahuta

30-March-2026

March 29, 2026

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Objective	3
2	System Design	3
2.1	Architecture Diagram	3
2.2	Workflow	3
3	Tools & Technologies	4
4	Code Snippets	4
4.1	Training ML Model	4
4.2	Data Generation	4
4.3	Pipeline and Training	4
4.4	Prediction Logic	5
4.5	Keyword Check	5
4.6	ML Model Prediction	5
4.7	Docker:	6
5	Application Preview	7
5.1	UI	7
5.2	Output	7
5.3	Working	9
6	Testing	9
7	Results & Discussion	10
8	Future Improvements	10

1 Introduction

1.1 Problem Statement

In this project, the objective is to create a mini AI-based application that predicts the sentiment of user input text as Positive, Negative, or Neutral using .NET and ML.NET.

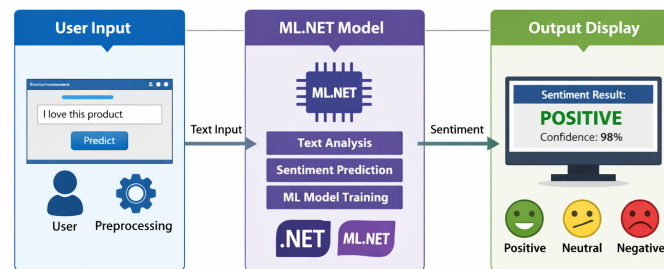
1.2 Objective

The application should:

- Accept user text input.
- Predict sentiment using a trained ML model.
- Display the sentiment clearly to the user.

2 System Design

2.1 Architecture Diagram



System Architecture: User Input → ML.NET Model → Output Display

Figure 1: System Architecture: User Input → ML.NET Model → Output Display

2.2 Workflow

1. User enters text in the Windows Forms TextBox.

2. Clicks the Predict button.
3. Input text is preprocessed.
4. ML.NET model predicts sentiment.
5. Result is displayed on the label.

3 Tools & Technologies

- **.NET Version:** .NET 4.8
- **Libraries:** Microsoft.ML, System.Windows.Forms
- **IDE:** Visual Studio 2022

4 Code Snippets

4.1 Training ML Model

The training process uses **ML.NET** to create a binary classification model that predicts sentiment (Positive or Negative).

4.2 Data Generation

- Positive and negative keywords are stored in lists.
- 100 sentences are generated for each sentiment using templates such as:
 - "I feel {word} today"
 - "This is {word}"
- Each sentence is labeled: **true** for positive, **false** for negative.

4.3 Pipeline and Training

- Text is converted into numerical features using **FeaturizeText**.
- SDCA Logistic Regression is used as the classifier.
- The pipeline is fitted to the training data and a **PredictionEngine** is created for real-time predictions.

```

private void TrainModel()
{
    mlContext = new MLContext();

    var trainData = mlContext.Data.LoadFromEnumerable(GetTrainingData());

    var pipeline = mlContext.Transforms.Text.FeaturizeText(
        outputColumnName: "Features",
        inputColumnName: nameof(SentimentData.Text))
        .Append(mlContext.BinaryClassification.Trainers
            .SdcaLogisticRegression(
                labelColumnName: nameof(SentimentData.Label),
                featureColumnName: "Features"));

    var model = pipeline.Fit(trainData);

    engine = mlContext.Model.CreatePredictionEngine<SentimentData, SentimentPrediction>(model);
}

```

4.4 Prediction Logic

The user enters text in the TextBox. The application predicts sentiment based on:

4.5 Keyword Check

- Input text is converted to lowercase.
- If it contains known positive words → predict **Positive**.
- If it contains known negative words → predict **Negative**.
- If neither → predict **Neutral**.

4.6 ML Model Prediction

- If both positive and negative keywords exist, the trained ML model is used to make the final decision.
- Sentiment is displayed along with prediction probability.

```

private void btnPredict_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(textBox1.Text))
    {
        MessageBox.Show("Please enter text!");
        return;
    }

    string inputText = textBox1.Text.Trim().ToLower();

    // Check if user input contains any of the training words
    bool containsPositiveWord = positiveWords.Any(w => inputText.Contains(w));
    bool containsNegativeWord = negativeWords.Any(w => inputText.Contains(w));

    if (!containsPositiveWord && !containsNegativeWord)
    {
        // Completely unknown words → Neutral
        lblMessage.Text = "Neutral 😊";
        return;
    }
}

```

4.7 Docker:

To containerize our .NET Sentiment Analysis application, we used a Dockerfile. Each line of the Dockerfile serves a specific purpose in building and running the application inside a Docker container:

- **FROM mcr.microsoft.com/dotnet/sdk:7.0**
This line specifies the base image for the container. We are using the official .NET 7.0 SDK image, which includes all necessary tools to build and run .NET applications.
- **WORKDIR /app**
This sets the working directory inside the container to `/app`. All subsequent commands (like copying files, building, and running the project) will be executed relative to this directory.
- **COPY . .**
This command copies all files and folders from the local project directory into the container's `/app` directory. This ensures that the container has the full project source code.
- **RUN dotnet build**
This builds the .NET project inside the container. The command restores dependencies and compiles the application, preparing it to run.
- **CMD ["dotnet", "run"]**
This defines the default command to execute when the container starts. It runs the application using the `dotnet run` command inside the container.

Using this Dockerfile allows our application to run in a consistent environment, independent of the host machine, which ensures that it works reliably on any system that supports Docker.

```
FROM mcr.microsoft.com/dotnet/sdk:7.0

# Set working directory inside container
WORKDIR /app

# Copy all project files into container
COPY . .

# Build the project inside container
RUN dotnet build

# Command to run the application
CMD ["dotnet", "run"]
```

5 Application Preview

5.1 UI

Form1
Home

5.2 Output

Positive
Negative

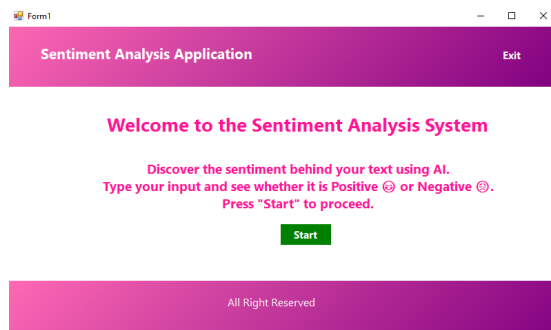


Figure 2: Windows Welcome Form UI Preview

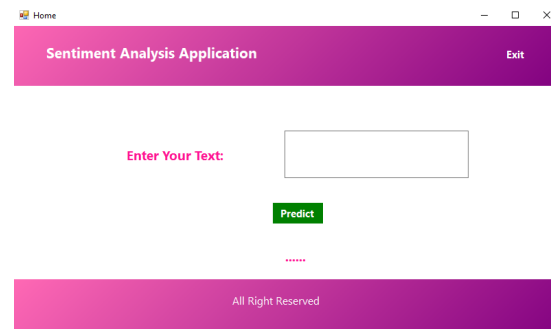


Figure 3: Windows Home Form UI Preview

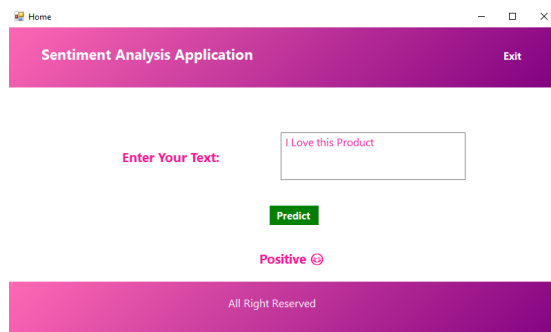
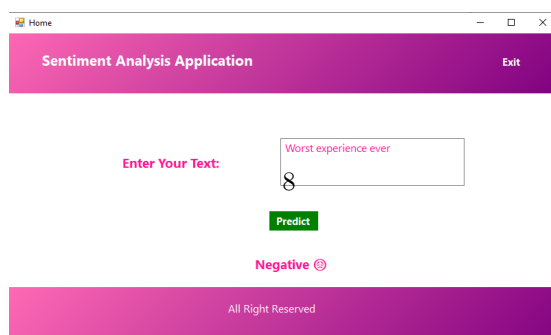
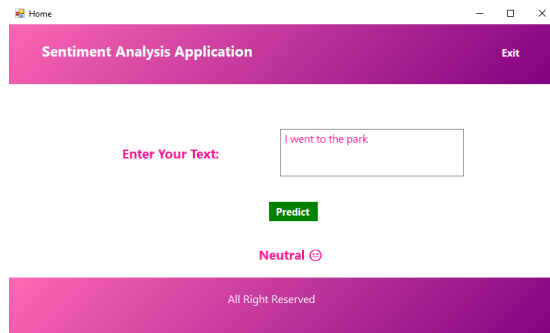


Figure 4: Positive Output Preview



Negative Output Preview

Neutral



Neutral Output Preview

5.3 Working

The Windows Forms application provides a simple UI:

1. **Text Input:** A TextBox for user input.
2. **Predict Button:** Triggers prediction logic.
3. **Output Label:** Displays one of three results: Positive, Negative, or Neutral, including probability for ML-based decisions.
4. **Exit Button:** Closes the application.

6 Testing

Input	Predicted Output
I love this product	Positive
This is terrible	Negative
I went to the park	Neutral
Absolutely fantastic app	Positive
Worst experience ever	Negative

Table 1: Sample Inputs and Outputs

7 Results & Discussion

- The AI model accurately predicts Positive and Negative sentiments for keyword-based sentences.
- Neutral is detected when no known keywords are present.
- Limitations include limited training data and inability to fully understand complex sentences.

8 Future Improvements

- Integrate API-based AI for better understanding.
- Expand dataset with real-world sentences.
- Add multi-class sentiment prediction.
- Improve UI with charts or visualizations.

THE END